Data Structure

Project Proposal

113306069	資管二	吳昀蓁
112510401	東印三	呂侑軒
112206027	經濟三	蔡依辰
111510407	東印四	賈雲翔

目錄

一、Introduction

(A) Topic 1

(B) Motivation 1

二、Search tricks

(A) Web Page Scoring and Ranking 1

(B) Multilingual Processing 3

三、System design

(A) Classes 4

(B) Class Diagram 10

四、Schedule

(A) WBS 10

(B) Task NetWork 12

五、Challenges

(A) Generative AI 13

(B) Google Related Keyword Search 13

一、Introduction

(A) **Topic:** Food

(B) **Motivation**

Our topic of interest was cakes initially, but considering that the scope was too narrow and could lead to bias during testing, we decided to broaden the topic to “food” in order to collect more diverse and representative data. Moreover, food is an element everyone is familiar with in daily life, allowing our project to be more closely connected to real-world applications. In addition, since NCCU is often jokingly referred to as a “food desert,” we hope that by combining this theme with the application of data structures, we can organize nearby dining information in a data-driven way, making our learning outcomes more relevant to everyday life.

二、Search tricks

(A) Web Page Scoring and Ranking

The system calculates the relevance between each web page and the search topic, then outputs the pages ranked by their scores in descending order.

The overall process is divided into the following three stages:

I. Recursive Web Page Expansion
The first result returned by the search engine is used as the starting page and serves as the root node of the website tree. The system then reads the hyperlinks on that page and adds successfully accessible links as child nodes, forming a tree structure composed of multiple web page nodes (the expansion is performed recursively downward). At each level, only a fixed number of links are selected to prevent excessive expansion. Crawling stops when the predefined maximum depth is reached. This establishes a website structure that is “topic-centered and outward-expanding.”

II. 2. Keyword Score Calculation
For successfully loaded web pages, content analysis and keyword scoring are performed.

Scoring procedure:

- Extract the plain text content of the web page.

- Convert all text to lowercase to facilitate keyword matching.
- Count the occurrences of each keyword.
- Multiply the occurrence count by the corresponding keyword weight and accumulate the score.

The score at this stage represents the relevance between an individual page and the search topic.

III. Result Sorting and Output

The system collects the URL, title, and score of all nodes, sorts them by score in descending order, and prioritizes pages with higher scores for output.
Pages that are more relevant to the input content and topic are displayed first.

Score Calculation Methods:

Postorder Traversal, weighted keyword scoring, and structural score accumulation.

I. Postorder Traversal Weighted Accumulation

Postorder Traversal is used to accumulate scores from child pages. When a child page's score is passed back to its parent page, a fixed decay factor (0.6) is applied.
Since parent pages are considered more important than deeper-level pages, the influence of deeper content decreases with depth.

Calculation steps:

- Compute the scores of all leaf-level child pages
- Propagate scores upward layer by layer, accumulating them as (child page score × 0.6)
- Finally, compute the overall score of the root node

II. Topic Relevance Evaluation

In addition to core search keywords, the system also considers multiple sets of pre-stored or LLM-provided keywords (e.g., food, restaurant, cuisine).

When a page contains both the core keywords and multiple topic-related keywords, its overall score increases significantly, indicating that the content better matches the search intent.
This approach simultaneously accounts for user search intent, semantic

completeness, and topic consistency, resulting in rankings that better reflect real-world search needs.

Score Weights:

Different weights are assigned based on keyword source and importance.

- User input (translated keywords) + automatically expanded translated keywords: 5.0
- Built-in system lexicon and Google related search terms: 1.0

(B) Multilingual Processing

Problem Description:
Web pages can be searched normally when the input is in English. However, when other languages are used (such as Chinese, Thai, Japanese, or Korean), garbled text may appear or relevant content may not be displayed.

Improvement Steps:
LLM-based multilingual translation with unified English keyword processing.
Through the LlmService, a large language model is called to convert user input in any language into English search keywords.

Input Processing:

I. Translation Rules
The system converts user input in any language into semantically accurate English keywords.The goal is not literal translation, but natural English that aligns with search engine understanding.

II. Noise Filtering
During translation, meaningless characters or symbols (e.g., @, #, !!!, or fragments unrelated to search intent) are automatically removed or ignored to prevent interference with keyword topic detection.

III. Keyword Expansion
To improve the completeness of search results, the system automatically appends 2–3 high-quality auxiliary keywords (e.g., *delicious*, *restaurant*, *best*) to the translated English keywords.
Even if a web page does not exactly match the core keyword, as long as its content belongs to a related topic (such as food reviews or restaurant introductions), it can still be effectively retrieved and identified.

IV. Non-topic Identification

When the user input is clearly unrelated to food content (e.g., brand or company names), the system automatically appends contextual hints.(e.g., input: *Tesla* ,processed as: *Tesla food*.)This prevents search results from completely deviating from the system's primary application scenario (food-related searches).

V. Garbled or Unrecognizable Input

If the user input consists of unrecognizable garbled text or meaningless strings, the system does not return an error directly. Instead, it applies a degradation strategy by randomly selecting a common and generic popular food keyword (e.g., *Pizza*, *Burger*, *Sushi*) as the search starting point.This ensures that the system can operate normally under all circumstances and avoids process interruption due to abnormal input.

三、System design

(A) Classes

I. Main.java

Description:

- The main entry point of the project (Main class).
- Starts the Spring Boot application using SpringApplication.run(...).
- Launches the embedded server (Tomcat), allowing the website to run at http://localhost:8080.
- Several Keyword instances (food, restaurant, delicious, cuisine) are created in the main method but are not ultimately used.

II. SearchService.java

Service Layer.
Description:

- Handles search logic and keyword analysis.
- Uses classes such as WebTree, WebNode, WebPage, and Keyword to generate the final ranked results.

Methods:

- search(): Calls GoogleQuery.queryByApiWithDetails() to retrieve complete search results including title, link, snippet, and date.

- searchAsMap(): Converts search results into a Map<title, link> format.
- searchWithCrawl(): Recursively crawls web pages starting from a specified URL (crawlNode), builds a WebTree structure, calculates node scores, and returns sorted results.

III. GoogleQuery.java

Description:

- External API handling class.
- Responsible for sending query requests to the Google Custom Search API.

Methods:

- queryByApiWithDetails(): Calls the Google API, parses JSON responses, and returns a List<Map<String, String>> containing title, link, snippet, and date. It also attempts to handle potential encoding issues using URLDecoder.
- fetchRelatedKeywords(): Uses Jsoup to crawl Google search result pages and extract related search suggestions (a.K8tyEc elements).
- fetchContent(): Uses Jsoup to connect to Google search pages and retrieve HTML content.

IV. RestConfig.java

Description:

- Spring Boot configuration class.
- Creates a RestTemplate Bean for use by GoogleQuery or other classes when calling external APIs.
- Supports dependency injection via @Autowired.

V. WebTree.java

Business Logic Core.
Description:

- Used to construct the tree structure of search results.
- Contains a root property (of type WebNode) representing the root of the tree.
- The constructor accepts a WebNode as the root node.
- Actual score calculations and tree operations are handled by WebNode and SearchService.

VI. WebNode.java

Description:

- Represents a node within the WebTree.
- Properties: parent(parent node), children(list of child nodes), webPage(the associated web page), nodeScore(node score).

Methods:

- addChild(): Adds a child node to the children list and sets its parent.
- isTheLastChild(): Determines whether this node is the last child of its parent.
- setNodeScore(): Recursively calculates node scores by first computing the webPage score, then accumulating scores from child nodes with a weight of 0.6.
- getDepth(): Calculates the depth of the node within the tree.

VII. WebPage.java

Business Object Layer.
Description:

- Encapsulates data for a single web page.
- Properties: url(page URL), name(page name), counter(WordCounter instance), score(page score).
- The constructor accepts url and name and automatically creates a WordCounter.

Method:

- setScore(): Calculates the page score by calling computeScore() with the provided list of Keyword objects.

VIII. Keyword.java

Description:

- Represents a keyword and its importance weight.
- Properties: name(keyword name), weight(keyword weight).
- Used in search score calculations, where the weight influences ranking results.

Methods:

- getName(): Returns the keyword name.
- getWeight(): Returns the keyword weight.

IX. WordCounter.java

Description:

- Responsible for fetching content from a specific URL and counting keyword occurrences.
- Property: root(WebNode type).

Methods:

- computeScore(): Uses Jsoup to retrieve plain text content from a web page, counts occurrences of each Keyword, multiplies them by their weights, and returns the total score.
- countOccurrences(): Counts the number of occurrences of a specific string within the text.

X. LanguageProcessor.java

Description:

- Language and keyword processing module.
- Originally designed to handle multilingual searches, storing food-related vocabulary in multiple languages and providing detectLanguage() to determine language based on Unicode ranges.
- This functionality has been largely replaced by AI translation in LlmService. Most content is currently unused, except for English-related keywords used in scoring.

Method:

- getFoodKeywords(): Returns a list of food-related keywords based on language code. It is called in /crawlSearch within SearchController for crawler scoring.

XI. LlmService.java

Description:

- AI language model service class (Service Layer).
  Uses the Google Gemini 2.5 Flash API for intelligent query optimization.
- translateToEnglishSearchTerm(). Accepts user input in any language and performs the following via Gemini AI:
  - Translation to English with whitespace and noise handling.
  - Intelligent keyword expansion (e.g., adding delicious, restaurant).

    - Non-food term handling (automatically appending food).
    - Garbled text protection (randomly generating popular food names such as .Pizza, Burger, Sushi).
    - Returns a safe default value "food" if the API call fails.
- Uses RestClient to send POST requests and parse JSON responses.

XII. index.html

Description:

- Frontend user interface page.
- Displays a search engine interface with an input field and search button.
- Uses JavaScript fetch('/search?q=xxx') to communicate with the backend.
- Displays a “Searching...” loading indicator during queries and supports Enter key search.
- Receives results in JSON format and displays:
    - Title (clickable link)
    - URL
    - Short snippet
    - Publication date
    - Score

XIII. application.properties

Description:

- Spring Boot configuration file.
- Configuration includes:
    - server.port (server port, default 8080)
    - google.cse.apiKey (Google Custom Search API key)
    - google.cse.cx (Google Custom Search Engine ID)
    - gemini.api.key (Google Gemini AI API key)

In addition, several classes exist in the project but are not actively used because their functionality has been replaced by other classes.

XIV. HTMLHandler.java

Description:

- Utility / data processing class.
- Property: root(WebNode type).
- Originally intended to handle HTML-related tasks (such as fetchHTML(), extractLinks(), and extractText()), but these responsibilities are now directly

handled by other classes (e.g., WordCounter, SearchService) using Jsoup. As a result, HTMLHandler has no practical functionality.

XV. FoodQueryNormalizer.java

Description:

- Service layer class.
- Responsible for query normalization and language processing.
- Its functionality has been replaced by AI translation in LlmService, and this class is not actively invoked.

Method:

- normalizeToFood(): Accepts user input, removes special symbols, detects the language, and appends corresponding food-related keywords (Chinese: “美食”, Japanese: “グルメ”, Korean: “맛집”, others: “food”).

XVI. Ranker.java

Description:

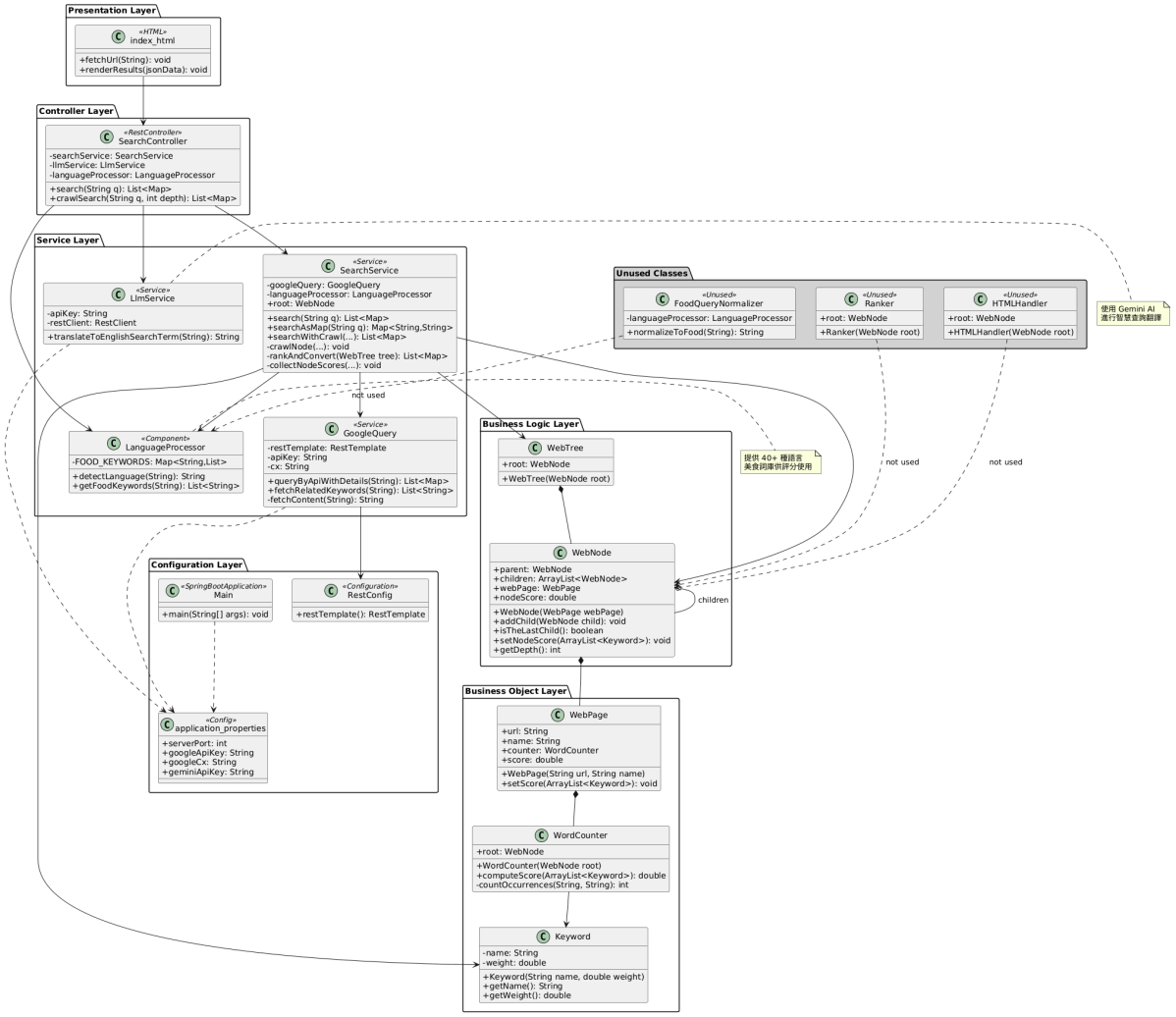
- Business logic helper class.
- Property: root (WebNode type)
- Originally intended to handle ranking logic, but the actual sorting functionality has been integrated into the rankAndConvert() method within SearchService.

XVII. SearchController.java

Controller Layer.
Description:

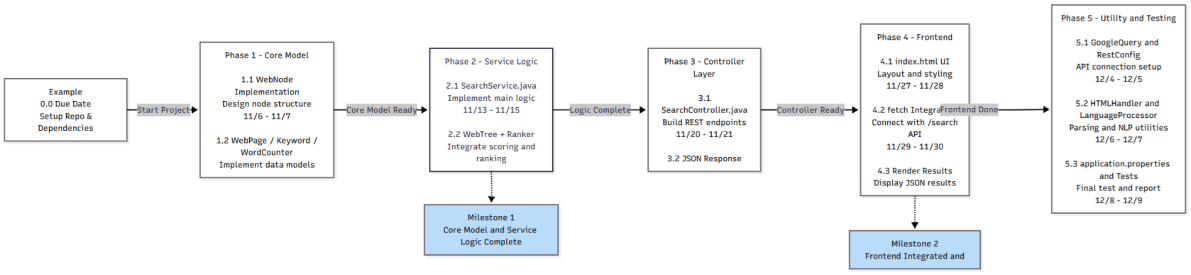
- Responsible for receiving frontend requests and invoking backend services.
- /search: Accepts a user query string, translates it into English via LlmService, then calls SearchService and returns search results in JSON format containing title, link, snippet, date, and score.
- /crawlSearch: An advanced search endpoint that accepts a query string and crawler depth parameter. After translation via LlmService, it performs web crawling and scoring, then returns a sorted result list.
- Integrates LlmService, SearchService, and LanguageProcessor to handle multilingual food search requirements.

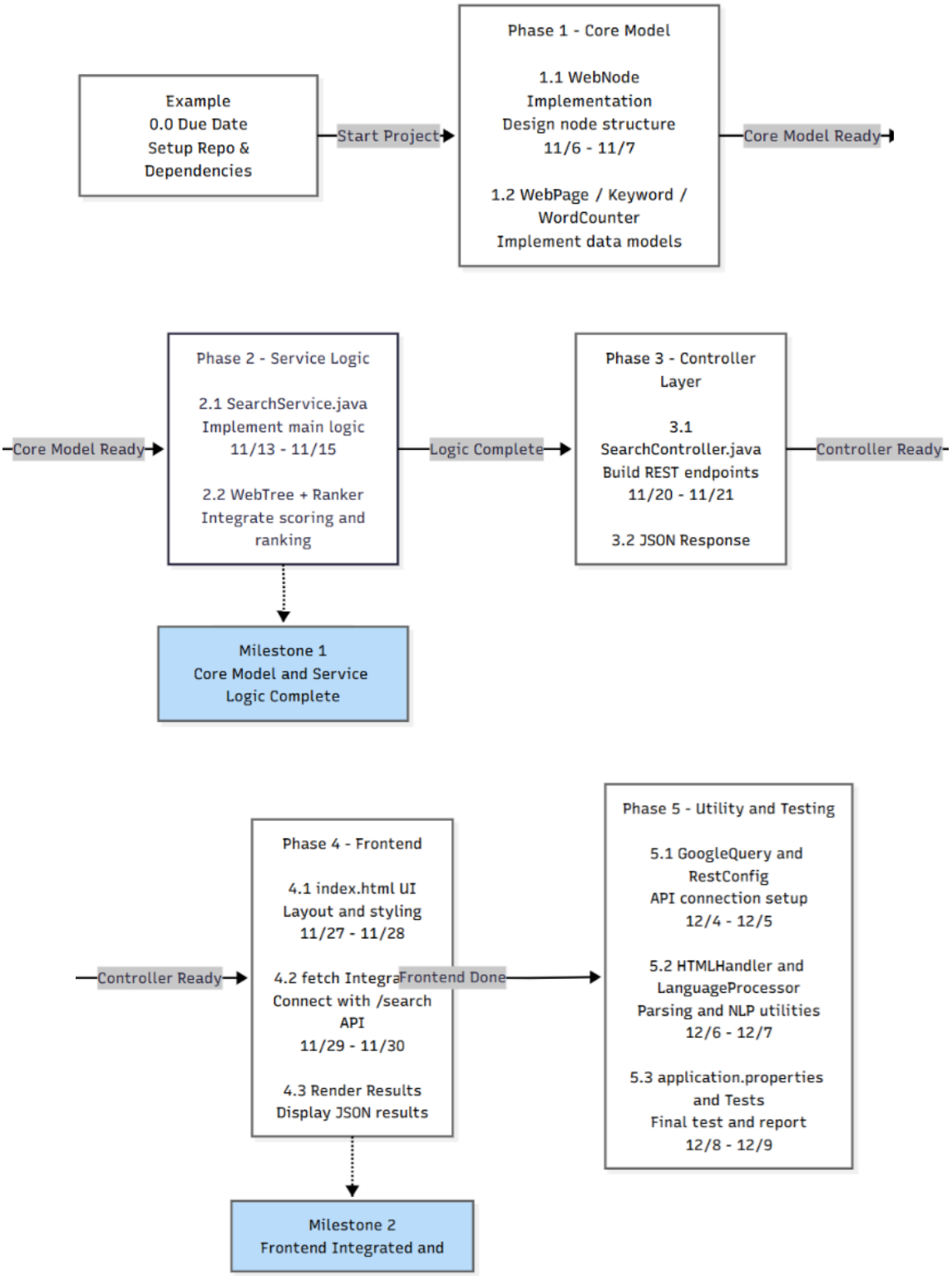
(B) Class Diagram



四、Schedule

(A) WBS




Example
0.0 Due Date
Setup Repo &
Dependencies
Start Project
Phase 1 - Core Model
1.1 WebNode
Implementation
Design node structure
11/6 - 11/7
1.2 WebPage / Keyword /
WordCounter
Implement data models
Core Model Ready
Core Model Ready
Phase 2 - Service Logic
2.1 SearchService.java
Implement main logic
11/13 - 11/15
2.2 WebTree + Ranker
Integrate scoring and
ranking
Logic Complete
Phase 3 - Controller
Layer
3.1
SearchController.java
Build REST endpoints
11/20 - 11/21
3.2 JSON Response
Controller Ready
Milestone 1
Core Model and Service
Logic Complete
Controller Ready
Phase 4 - Frontend
4.1 index.html UI
Layout and styling
11/27 - 11/28
4.2 fetch Integra
Connect with /search
API
11/29 - 11/30
4.3 Render Results
Display JSON results
Frontend Done
Phase 5 - Utility and Testing
5.1 GoogleQuery and
RestConfig
API connection setup
12/4 - 12/5
5.2 HTMLHandler and
LanguageProcessor
Parsing and NLP utilities
12/6 - 12/7
5.3 application.properties
and Tests
Final test and report
12/8 - 12/9
Milestone 2
Frontend Integrated and

(B) Task NetWork

```
FoodSearch Application
│
├── Frontend (Presentation Layer)
│   ├── index.html UI
│   ├── Fetch API Integration
│   ├── Loading State Display
│   └── Render Results
│
├── Controller Layer
│   ├── SearchController
│   │   ├── /search
│   │   └── /crawlSearch
│   └── JSON Response
│
├── Service Logic Layer
│   ├── SearchService
│   │   ├── Google API Integration
│   │   ├── WebTree Construction
│   │   ├── Recursive Crawling
│   │   └── Score Ranking
│   ├── LlmService
│   │   ├── Gemini AI Integration
│   │   ├── Multi-language Translation
│   │   └── Query Optimization
│   └── GoogleQuery
│       ├── Custom Search API
│       ├── Result Details Extraction
│       └── Related Keywords Fetch
│
├── Business Logic Layer
│   ├── WebTree
│   ├── WebNode
│   │   ├── addChild()
│   │   ├── setNodeScore()
│   │   └── getDepth()
│   └── WebPage
│       └── setScore()
│
├── Business Object Layer
│   ├── Keyword
│   └── WordCounter
│       ├── computeScore()
│       └── countOccurrences()
│
├── Utility & Config Layer
│   ├── LanguageProcessor
```

```
│   │   ├──40+ Language Food Keyword Lexicon
│   │   ├── detectLanguage() (Not used)
│   │   └── getFoodKeywords()
│   ├── RestConfig
│   └── Main
│
├── Configuration
│   └── application.properties
│       ├── server.port
│       ├── google.cse.apiKey
│       ├── google.cse.cx
│       └── gemini.api.key
│
└── Unused Classes
    ├── FoodQueryNormalizer
    ├── Ranker
    └── HTMLHandler
```

五、Challenges

(A) Generative AI

One optional challenge for this project is integrating generative AI into our pipeline — specifically connecting models such as Gemini and ChatGPT via their APIs. This will be treated as a stretch goal to pursue only if we have the time and resources.

(B) Google Related Keyword Search

After the user enters any keyword, the system not only performs a standard Google search, but also extracts the “Related Searches” keywords provided on the Google search results page and incorporates them into subsequent web page analysis and scoring mechanisms.By leveraging the related search keywords provided by Google, the relevance of web page content is further enhanced.

Process:

I. Extracting Google “Related Searches” section.
II. Retrieve the HTML content of the Google search results page.
III. Use an HTML parsing tool (Jsoup) to analyze the page structure.
IV. Locate the terms corresponding to the “Related Searches” section on the search results page and store them as a list of related keywords for the search results.

Organizing and Integrating Related Search Keywords:

I. The system performs basic cleanup on the extracted Google related search keywords (e.g., avoiding null or empty values).

Finally, the Google related search terms are included in the scoring mechanism.